

## CS 744: Design and Engineering of Computing Systems Autumn 2026 Lab 2: system calls, the unsung heroes

### Instructions

- This course is on a **no-plagiarism** diet. All parties involved in plagiarism harakiri will be referred to DADAC and penalized to the maximum extent.
- The Moss Detective Agency has agreed to conduct all investigations.  
<https://theory.stanford.edu/~aiken/moss/>  
Byomkesh, Sherlock, Phryne, Marple, and Hercule are on standby.
- Hardcoding the expected output of execution in the source code will yield **negative** marks.
- Generative AI (ChatGPT, Gemini, etc.) is not you!  
Analytical, critical, and creative thinking are learning outcomes of this course, and source code or any content via GenAI tools cannot be part of your submissions.
- **Note:** Submission guidelines to be strictly followed; otherwise, your submission will not count.
- Log in with username your **CSE LDAP ID** or **labuser** on the software lab machines for this lab or on your look Linux environment/setup.
- This file is part of the **lab1.tar.gz** archive, which contains multiple directories with programs associated with the exercise questions listed below.
- Most questions in this lab are tutorial-style and aim to provide an introduction to tools and files related to system and process information and control.

---

### Task 0: background

Refer to the **man** pages for the following system calls —

**fork, exec, wait, waitpid, exit, open, dup, dup2, pipe, getpid, getppid**

A few things to note -

- A **zombie** process is a process that has exited but not yet been reaped by its parent process via the **wait/waitpid** system call.
- The **open(const char\* path, int flags)** system call opens the file specified by *path*. If the specified file does not exist, it may optionally be created (if **O\_CREAT** is specified in *flags*). The return value of **open** is a non-negative integer that is an index to an entry file descriptor table in the process PCB. The file descriptor is an indirection/reference to the open file, used with other systems for file operations.
- The **dup(int oldfd)** system call allocates a new file descriptor that refers to the same open file description as the descriptor *oldfd*. After **dup**, two descriptors refer to the same file. The new file descriptor number is guaranteed to be the lowest unused file descriptor number in the calling process.
- The **dup2(int oldfd, int newfd)** system call performs the same task as **dup()**, but instead of using the lowest numbered unused file descriptor, it uses the file descriptor number specified by *newfd*. In other words, the file descriptor *newfd* refers to the same open file as the file descriptor *oldfd*.

## Task 1: how to use a fork?

The **fork** system call is used to create a new process by duplicating the process that invokes fork (the parent process). `fork()` produces a second, independent process (the child process) that starts out as an almost exact copy of the calling process (the parent): same code, same variable values, same open file descriptors, but a separate address space and a new PID. Both the parent and child processes resume execution at the very next statement after the `fork()` call, which means every line of code after the fork runs in both processes unless you branch on the return value.

The only way to tell parent process and child process apart is **fork()'s** return value, which differs depending on who's asking: in the parent, `fork()` returns the child's PID (a positive integer); in the child process, it returns 0; if the call fails (for instance, the system has no resources left to create another process), no child is created at all, and `fork()` returns -1 to the original, unchanged calling process.

The child process runs on its own now, so the parent doesn't automatically find out when it's done — it has to ask. That's what `wait()/waitpid()` do: they pause the parent until a child process finishes, and then tell the parent how that child process exited.

Refer to the man pages of the following syscalls:

`fork()`, `wait()`, `waitpid()`, `getpid()`, `getppid()`, `exit()`

### A. `fork()`

Write a program **fork.c** in which the parent process creates a child process using `fork()`. The parent process should print its role ("Parent process") and PID, and the child should print its role ("Child process"), PID, and its parent's PID, then exit.

**Sample Output:**

```
Shell
$ gcc fork.c -o fork

$ ./fork
Parent process 45780 , child PID: 45781
Child process 45781 , parent PID: 45780
```

## B. wait, the reaper!

Extend the previous program `fork.c` to **`reap_child.c`**, where the parent process creates one child using `fork()`. The child should print its role and PID, sleep for 2 seconds, and exit normally. The parent process must wait for the child using `wait()` before printing a completion message.

### Sample Output:

```
Shell
$ gcc reap_child.c -o reap_child

$ ./reap_child
Parent process 48087 , child PID: 48088
Child process 48088 , parent PID: 48087
Child process 48088 has finished
```

## C. what is the child process up to?

Write a program **`child_status.c`** where the child exits with status code **10** (or any integer) using `exit(10)`. The parent process should use `wait()` to retrieve the child's termination status and print the exit code using `WEXITSTATUS()`.

### Sample Output:

```
Shell
$ gcc child_status.c -o child_status

$ ./child_status
Parent process 50934 , child PID: 50935
Child process 50935 , parent PID: 50934

Child process has terminated.
Child process Exit Status = 10
```

## D. reaping named (numbered) child processes

Write a program **ordered\_wait.c** that creates two children.

- Child process 1 sleeps for 5 seconds.
- Child process 2 sleeps for 2 seconds.

The parent process must reap the child processes in the order:

Child process 1 → Child process 2 using `waitpid()`.

### Sample Output:

```
None
$ gcc ordered_wait.c -o ordered_wait

$ ./ordered_wait
First child process 53882 , parent PID: 53881
Second child process 53883 , parent PID: 53881

First Child process has terminated.
First Child process Exit Status = 160

Second Child process has terminated.
Second Child process Exit Status = 64
```

## E. non-blocking reaping: wait without waiting

You are given the following incomplete program (**reap\_children.c**). The program creates 100 child processes, each of which sleeps for a random duration before exiting.

The program creates 100 child processes, each of which runs for a random amount of time before terminating. The parent periodically checks for terminated children using `waitpid()` with the `WNOHANG` option, reaps them as they finish, and sleep for some time if no process is reaped, and continues until all child processes have been reaped.

Your task is to complete the missing parts marked with **TODO**.

### Fill in the missing code for the following:

1. Detects whether the current process is the child process after `fork()`.
2. Write the condition that keeps the parent process running until all child processes have been reaped.
3. Use `waitpid()` with the `WNOHANG` option to check for terminated child processes.
4. Verify that a child process exited normally before printing its exit status.

## Questions

1. What does `waitpid(-1, &status, WNOHANG)` return in each of the following cases?
  - A child process has terminated.
  - No child process has terminated.
  - There are no remaining child processes.
2. What would happen if the parent process never called `wait()` or `waitpid()`?
3. Why is the first argument to `waitpid()` set to `-1`?
4. What status does the parent process receive when a child exits with a status  $> 255$ , and why?

## Task 2: is fork enough for world peace?

The `fork()` system call creates a copy of an existing process. However, simply replicating the same work cannot achieve goals like world peace; it requires different kinds of efforts working together. The `exec()` family of system calls makes this possible by replacing the current process with a new program, allowing each process to perform a different task. Unlike `fork()`, which creates a new process, `exec()` does not create another process; instead, it loads a new executable into the existing process's memory and begins execution from the new program's `main()` function. If the `exec()` call succeeds, the statements that follow it are not executed because the original program has been completely replaced. If an error occurs (for example, if the executable file does not exist), `exec()` returns `-1`, allowing the program to handle the error appropriately.

Refer to the man pages of the following syscalls:

`fork()`, `wait()`, `exec()` / `execvp()` / `execlp()`

### A. `exec(ute)` a command

Write a program, `cmd_exec_a.c`, that executes the Linux command `ls -l` using an appropriate variant of the `exec()` system call. Before executing the command, the program should print its Process ID (PID). If the command cannot be executed, the program should print execution failed and terminate gracefully.

**Sample Output:**

```
Shell
$ gcc cmd_exec_a.c -o cmd_exec_a
$ ./cmd_exec_a
cmd_exec PID: 14523
total 8
-rwxr-xr-x cmd_exec_a
-rw-r--r-- cmd_exec_a.c
```

### B. `exec(ute)` with arguments

Create a new program `cmd_exec_b.c`, to execute a Linux command provided through the command-line arguments. Your program should accept the command and its arguments, print its Process ID (PID). If no command is provided, display an appropriate usage message. If the command cannot be executed, print "execution failed" and terminate gracefully. You should also support execution with multiple arguments, for e.g. "echo Hello World"

**Sample Output:**

```
Shell
$ ./cmd_exec_b ls -l
cmd_exec PID: 23451
total 8
-rwxr-xr-x cmd_exec_b
-rw-r--r-- cmd_exec_b.c
$ ./cmd_exec_b echo Hello World
cmd_exec PID: 23467
Hello World
$ ./cmd_exec_b invalid_command
cmd_exec PID: 23481
Execution failed
```

**Note:**

You can use `perror()` for printing concise error messages. Read more in man pages.

### C. `exec(ute)` with a `fork()`

Create a new program, **`cmd_exec_c.c`**, that executes a Linux command inside a child process. The idea here is to create a child process first using `fork()`, and then execute the command provided via the command-line arguments using the appropriate `exec()` system call in the child process. This allows the parent process to wait for the child process to complete and reap it before terminating. As with earlier tutorials, handle all error conditions gracefully, including failures of both `fork()` and `exec()`.

**Sample Output:**

```
Shell
$ ./cmd_exec_c ls
Parent PID: 31240
Child PID: 31241
cmd_exec_c
cmd_exec_c.c
Child process completed successfully.
$ ./cmd_exec_c invalid_command
Parent PID: 31258
Child PID: 31259
Execution failed
Child process terminated.
```

## the file descriptors detour

References: [link1](#) [link2](#)

In UNIX-based OSes, each process has a **per-process file descriptor table** as part of the OS state to keep track of all the open files of the process (The fd table is an array of pointers to the global file table, and the fd is an index to an array element in this array). Further, each entry in the file descriptor table points to a global file table. The **global** file table contains actual metadata about the file (an example would be where the file is actually located on disk, the current offset for file operations, the number of file descriptor references to the file, etc.).

Entries at index 0, 1, and 2 in each process's file descriptor table normally point to the **STDIN** or **standard input** (input from your terminal/console), **STDOUT** or **standard output** (output to your terminal/console), and **STDERR** or **standard error** (output to your terminal/console) files, respectively. These file descriptors are known as standard file descriptors.

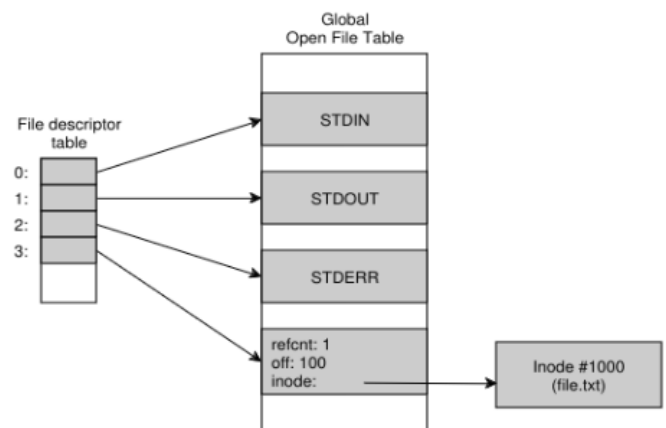
But why? Why do standard file descriptors exist in UNIX systems' processes, and where are they Typically used in a program's execution?

As it turns out, the first process that is created (init process) opens these files, and then these file descriptors are inherited from parent processes upon `fork()`. Interestingly, all interactions that are done with routines such as `printf()`, `scanf()`, which in turn use the write and read system calls, happen through these files.

Consider the following sample program:

```
int main(){
    char buffer1[100];
    int fd = open("file.txt", ORDONLY);
    read(fd, buffer1, 100);
    close(fd);
    return 0;
}
```

The diagram shows the state of the file descriptor table and the global file table after the read system call has been performed and before the file is closed using the close system call.



## Task 3: write club (first rule: always check the return values)

Refer to the man pages of the following syscalls:

`open()`, `close()`, `read()`, `write()`, `dup()`, `dup2()`

### A. duplicating standard input and output using `dup()`

The `dup()` system call creates a duplicate of an existing file descriptor. The duplicated file descriptor refers to the same open file description as the original and can be used interchangeably.

Write a program named **`dup_stdio.c`** that should duplicate the file descriptors corresponding to **standard input** (`STDIN_FILENO`) and **standard output** (`STDOUT_FILENO`). After creating these duplicate FDs, use the duplicated standard input file descriptor to read data from the user and the duplicated standard output file descriptor to write the same data back to the terminal. The program should continue reading and echoing the user's input until an end-of-file (EOF) condition is encountered (press **Ctrl+D** on the terminal). Before terminating, ensure that the duplicated file descriptors are properly closed. Handle the errors gracefully.

Do not use `scanf()` and `printf()` for this question. Use `read()` and `write()` syscalls.

#### Sample Output:

```
Shell
$ gcc dup_stdio.c -o dup_stdio

$ ./dup_stdio
hello world!
hello world!
cs744
cs744
```

## B. redirecting standard input and output using `dup2()`

The `dup2()` system call duplicates an existing file descriptor (`oldfd`) to a target file descriptor (`newfd`). It is primarily used for I/O redirection (e.g., sending output to a file instead of the console).

Write a program named `dup2_redirect.c` to redirect the standard output to a file using `dup2()`. Your program should create (or open, if it already exists) a file named `dup2.txt`. Redirect the program's standard output (`STDOUT_FILENO`) to this file using the `dup2()` system call. After the redirection, write a `printf` statement "Hi there, Welcome to cs744." After writing the message, close the file descriptor and terminate the program.

Since the standard output has been redirected, this message should be written to `dup.txt` rather than displayed on the terminal.

## C. implementing a simple cat command

Write a program named `simple_cat.c` that implements a simplified version of the `cat` command. The program should create a child process using the `fork()` system call. The child process should read data using the `read()` system call from `STDIN` and write it to `STDOUT` using `write()` syscall. Do not use library functions such as `printf()`, `scanf()`, or the shell's `cat` command.

If the program is executed without any command-line arguments, it should read input from the keyboard (standard input) until the user presses Enter (a newline character), and then display the entered text on the terminal.

If the program is executed with one command-line argument, treat the argument as the name of an input file. The child process should open the file, read its contents to the end, and display them on the terminal. The parent process should wait for the child process to complete before terminating.

### Sample Output:

```
Shell
$ gcc simple_cat.c -o simple_cat

$ ./simple_cat
hello world!
hello world!

$ ./simple_cat cs744.txt
Welcome to CS744
```

## Task 4: shell we dance?

A shell is a program that acts as an interface between the user and the operating system. It reads commands from the user, creates child processes, and executes them. It does hand-holding for user processes to run on a machine. Your task here is to start building a simple shell using the understanding of the tasks you have done till now

### A. one step at a time

Using the provided base program **shell.c**, implement a basic Linux shell. The base program already repeatedly reads a line of user input and tokenizes it into individual words. Extend the program to accept user commands, create a child process using `fork()`, and execute the command in the child process with an appropriate `exec()` system call. The parent process should wait for the child process to complete before accepting the next command.

Your shell should support commands with multiple arguments and options (e.g., `ls -l`, `mkdir`, `echo`, etc.). If the command cannot be executed, print an appropriate error message and continue accepting further commands without terminating the shell.

Implement the built-in `exit` command so that typing it terminates the shell gracefully.

#### Sample Output:

```
Shell
$ gcc shell.c -o shell
$ ./shell
$ pwd
/home/student
$ ls -l
total 24
-rwxr-xr-x shell
-rw-r--r-- shell.c
$ echo Hello Operating Systems
Hello Operating Systems
$ invalidcmd
Execution failed
$ mkdir test
$ exit
```

#### Hint:

The tokenizer returns an array of strings that can be passed directly to `execvp()` as the argument list.

## B. serialised execution

Command1 && command2 - runs command2 only when command1 exited with status 0.

*Hint: Split the line on && first, run each command in order, and only continue to the next one if the previous command's exit status was 0, stop the chain as soon as one command fails.*

### Sample Output:

```
Shell
$ ./shell
$ false && echo skipped
[exit status: 1]
$ true && echo not skipped
[exit status: 0]
not skipped
[exit status: 0]
$ exit
exiting
```

## C. redirect the output

Add support for the primitives < file or > file redirects the STDIN or STDOUT using open() and dup2()

*Hints: Use open() to get a file descriptor for the filename. Use dup2(fd, STDIN\_FILENO) or dup2(fd, STDOUT\_FILENO) to point fd 0/1 at that file, then close() the original fd.*

### Sample Output:

```
Shell
$ ./shell
$ wc -l < file.txt
1
[exit status: 0]
$ wc file.txt
1 1 6 file.txt
[exit status: 0]
$ echo bye world > file.txt
[exit status: 0]
$ cat file.txt
bye world
[exit status: 0]
```

## D. background matters

Support a trailing & on the command line. If present, the shell must not wait for the child process to finish; it should immediately reprompt. Since the parent no longer waits for the child process synchronously, add `waitpid` so that background children never become zombies (*Take a hint from Task 1 part E*).

### Sample Output:

```
Shell
$ ./shell
$ echo before
before
[exit status: 0]
$ sleep 5 &
[background] started pid 41554
$ echo after
after
[exit status: 0]
$
[background] pid 41554 done (exit status 0)

$ exit
exiting
```

## Submission Guidelines

- All submissions via Moodle.
- Place all the files in your submission directory, with the directory name being your roll number.
- Tar and gzip the directory using the command

```
$ tar -zcvf <rollno_lab2>.tar.gz <rollno_lab2>
```

For example, if your roll number is 123456789, then the directory name will be **123456789**, and submission will be **123456789.tar.gz**, and if your roll number is 12D345678, then the directory name will be **12D345678**, and submission will be **12D345678.tar.gz**

- The tar should contain the following files in the following directory structure:

```
<rollnumber_lab2>/
|__task1/
|    |__fork.c
|    |__reap_child.c
|    |__child_status.c
|    |__ordered_wait.c
|    |__reap_children.c
|__task2/
|    |__cmd_exec_a.c
|    |__cmd_exec_b.c
|    |__cmd_exec_c.c
|__task3/
|    |__dup_studio.c
|    |__dup2_redirect.c
|    |__simple_cat.c
|__task4/
|    |__shell.c
```

- **Deadline: Xth August 2026, 5 pm. (via Moodle)**
-

[OPTIONAL]

## pipes — plumbing for the flow of information between processes

A pipe is a mechanism for inter-process communication using file descriptors.

The man page of the pipe system call is [here](#).

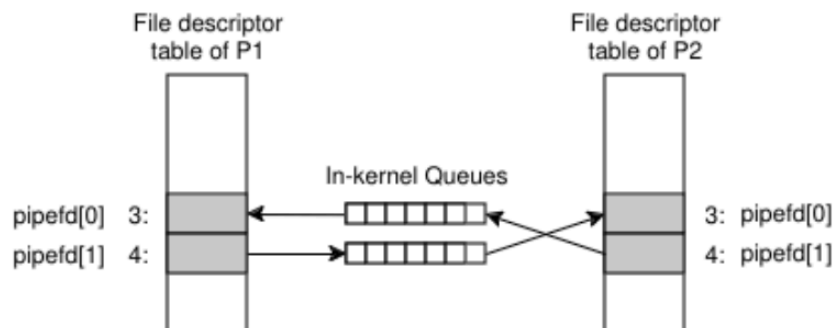
```
int pipe(int pipefd[2]);
```

The above call initializes a pipe and allocates two file descriptors using the arguments passed to it.

The pipe system call initializes the 0th file descriptor `pipefd[0]` as the read end of the pipe and `pipefd[1]` as the write end of the pipe, as shown in the figure.

After the pipe is set up, all writes using `fd[1]` are connected to an in-kernel pipe (i.e., queue) and can be read using the `fd[0]` file descriptor.

When used along with the `fork` system call, pipes enable Inter-Process Communication (IPC).



## A. using pipes

Write a program `pipe_a.c` that creates two child processes and connects the standard output of the first child to the standard input of the second child using the `pipe()` system call. The first child should take input from the standard input and find the square of the value, which should be written to the standard output (the pipe's write end). The second child should take the squared value from the standard input (the pipe's read end), find the square root of the same, and print the original value in the standard output.

### Sample Output:

```
Shell
user@users-linux:~$ ./pipe_a
Process A (pid: 1234): Input value of x: 4 Process B (pid: 1235): Got value from
A: 16 Process B (pid: 1235): Original value of x:4
```

## B. pipelining the file

Write a program to process the file `bigdata.txt` and determine how many times the following words appear, regardless of the case: **the**, **of**, **and**, **a**, and **to** in the data file.

A basic approach would be to write a program that sequentially scans the entire file and counts each word's occurrences. However, a more efficient method involves parallel processing using multiple processes. Write a program named `pipe_b.c` that takes a number as a command-line argument to specify the number of parallel processes the main process will set up to process the data file (`bigdata.txt`). Each child process works on a part of the file; if there are  $n$  child processes, each is assigned  $1/n$  of the data file for processing.

The parent process, which part of the data file a child process is responsible for, via a dedicated pipe, by sending a message in the following format:

**<start-offset>,<size of data chunk>**

For example, if there are two child processes and the file is 10 bytes long, the message format for each of the child processes will be -

**0, 5 – for one child process and  
5, 5 – for the second child process**

Each child process will seek its designated position in the file and read the assigned portion to perform the word count. Each process should maintain a histogram for the occurrences of the specified words. Once the computation is complete, the child processes should send the computed values back to the parent process through the same pipe.

A child process will convey the count values for each of the words in the following format:

**<the-count>,<of-count>,<and-count>,<a-count>,<to-count>**

After receiving the results from all child processes, the parent process should aggregate the word counts and display the final histogram. Additionally, the program should measure and print the total time taken for the computation. Experiment with different numbers of child processes (1, 2, 4, 8) and observe the impact on performance.

**All inter-process communication must be done using the pipe system call.**

**Sample Output:**

```
Shell
user@users-linux:~$ ./pipe_b 2
the: 4354
of: 6235
and: 1234
a: 5568
to: 7824
Computation time: 12s
```

*Note: If a process tries to read before something is written to the pipe, the process is blocked until something is written and is available in the pipe.*